

Maven

Why do we need Maven?

- If you are working on Java projects then most of the time you need dependencies.
- Dependencies are nothing but libraries or JAR files. You need to download and add them manually. Also, the task of upgrading the software stack for your project was done manually before Maven. So there was a need for a better build tool that would handle such issues.
- This where Maven comes into the picture.
- Maven can solve all your problems related to dependencies. You just need to specify the dependencies and the software version that you want in pom.xml file in Maven and Maven will take care of the rest.

What is Maven?

- The Maven project is developed by Apache Software Foundation where it was formerly a part of the Jakarta project.
- Maven is a powerful build automation tool that is primarily used for Java-based projects. Maven helps you tackle two critical aspects of building software –
- It describes how software is built
- It describes the dependencies.

- Maven prefers convention over configuration. Maven dynamically downloads Java libraries and Maven plug-ins from one or more repositories such as the Maven Central Repository and stores them in a local cache.
- The artifacts of the local projects can also be updated with this local cache.
- Maven can also help you build and manage projects written in C#, Ruby, Scala, and other languages.
- Project Object Model(POM) file is an XML file that contains information related to the project and configuration information such as dependencies, source directory, plugin, goals, etc. used by Maven to build the project.
- When you execute a maven command you give maven a POM file to execute the commands. Maven reads the pom.xml file to accomplish its configuration and operations.

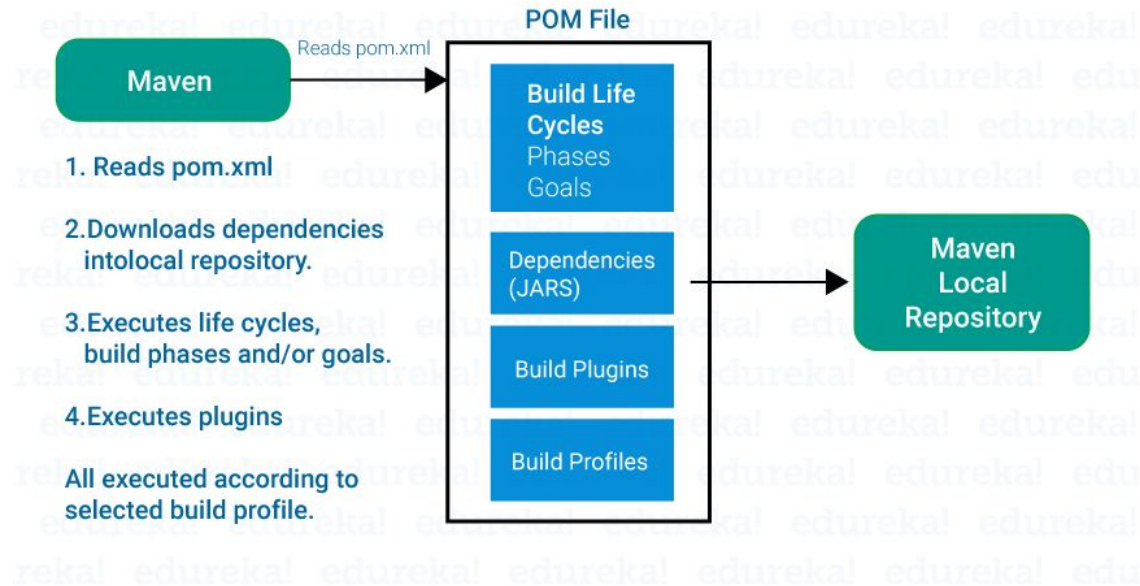
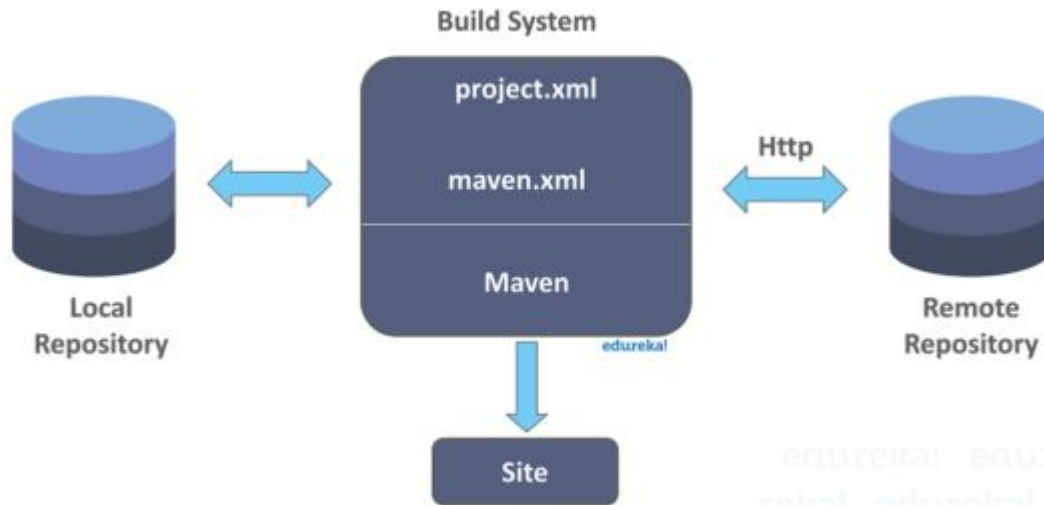
Maven Objectives



When should someone use Maven?

- If there are too many dependencies for the project.
- When the dependency version update frequently.
- Continuous builds, integration, and testing can be easily handled by using maven.
- When one needs an easy way to generate documentation from the source code, compiling the source code, packaging compiled code into JAR files or ZIP files.

Maven Architecture



Maven life cycle, phases and goals

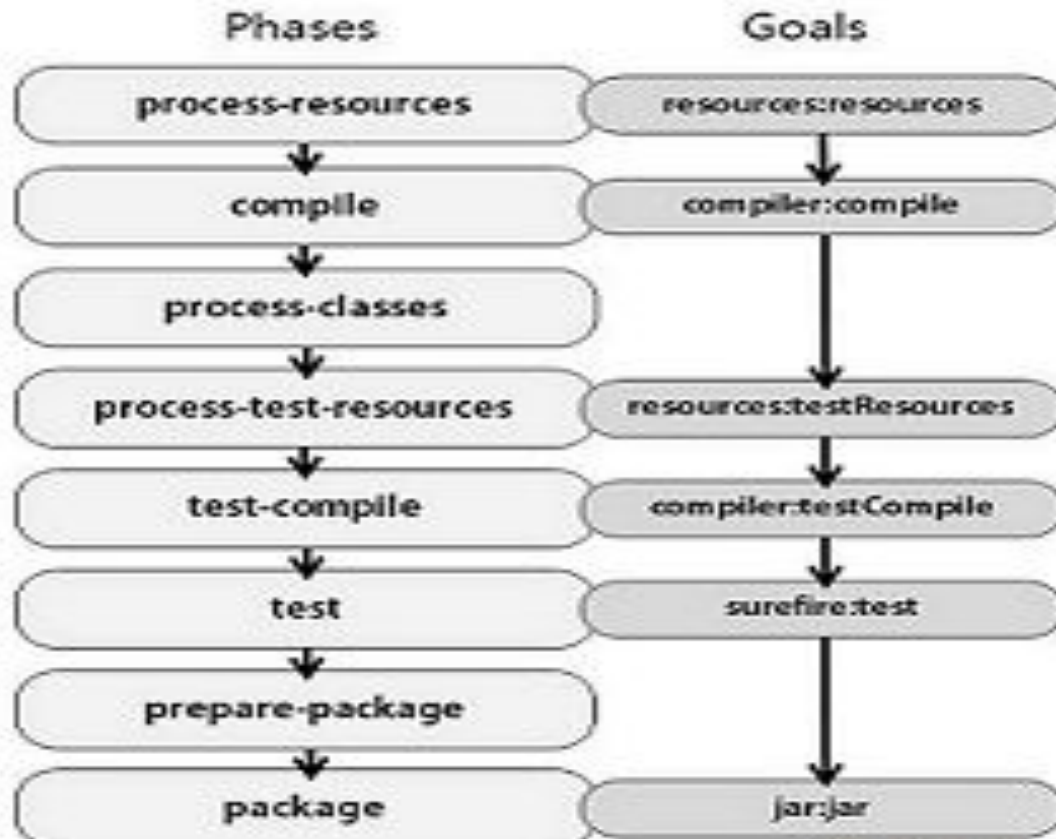
- Maven life cycle

Clean Lifecycle	Default Lifecycle	
pre-clean	validate	test-compile
clean	initialize	process-test-classes
post-clean	generate-sources	test
	process-sources	prepare-package
	generate-resources	package
	process-resources	pre-integration-test
	compile	integration-test
	process-classes	post-integration-test
	generate-test-sources	verify
	process-test-sources	install
	generate-test-resources	deploy
	process-test-resources	

- There is a specific life cycle that Maven follows to deploy and distribute the target project.
- There are three built-in life cycles:
- **default** – This is the main life cycle of Maven as it is responsible for project deployment.
- **clean** – This life cycle is used to clean the project and remove all files generated by the previous build.
- **site** – The aim of this life cycle is to create the project's site documentation.
- Each life cycle is made up of a sequence of phases. The default build life cycle consists of 23 phases as it is the main build life cycle of Maven
- On the other hand, clean life cycle consists of 3 phases, while the site life cycle is made up of 4 phases.

Maven Phases

- A Maven phase is nothing but a stage in the Maven build life cycle. Each phase executes a specific task.



- **validate** – This phase checks if all information necessary for the build is available
- **compile** – This phase compiles the source code
- **test-compile** – This phase compiles the test source code
- **test** – This phase runs unit tests
- **package** – This phase packages compiled source code into the distributable format (jar, war)
- **integration-test** – This phase processes and deploys the package if needed to run integration tests
- **install** – This phase installs the package to a local repository
- **deploy** – This phase copies the package to the remote repository

- Maven executes phases in a specific order. This means that if we run a specific phase using the command such as `mvn <phase>`, this won't only execute the specified phase but all the preceding phases as well.
- For example, if you run the command `mvn deploy`, that is, the deploy phase which is the last phase in the default build life cycle, then this will execute all phases prior to the deploy phase as well.

3. Maven Goals

- A sequence of goals constitutes a phase and each goal executes a specific task. When you run a phase, then Maven executes all the goals in an order that are associated with that phase. The syntax used is `plugin:goal`. Some of the phases and the default goals bound to them are as follows :
- `compiler:compile` – compile phase
- `compiler:test` – test-compile phase
- `surefire:test` – test phase
- `install:install` – install phase
- `jar` and `war:war` – package phase
- A Maven plugin is a group of goals. However, these goals aren't necessarily all bound to the same phase. For example, the Maven Failsafe plugin which is responsible for running integration tests. For unit testing, you need Maven surefire plugin.

Advantages and disadvantages of Maven:

- Starting with the advantages:
 1. Maven gives an easy start to the project in different environments.
 2. You can use Maven to easily build your project to a jar, war as per your requirements.
 3. You can easily add new dependencies by writing the dependency code in the pom file.
- Now, the disadvantages:
 1. If the maven code for an existing dependency is not available, then one cannot add that dependency.
 2. Maven installation along with plugins is a must.

Maven Repository:

- A repository is simply a directory on your machine. The project jars, plugins or any other project-related materials are stored here.
Let me elaborate on the different types of repositories.
 1. Local Repository
 2. Central Repository
 3. Remote Repository
- Whenever any dependency has to be searched, it is done in repositories. Maven initializes its search from Local repository, then Central repository and finally in Remote repository.

- **Local Repository:**
- The local repository of Maven is a folder location on your machine, where all the project related elements are stored. As soon as the Maven build is executed, Maven automatically downloads all the dependency jars into the local repository.
By default, maven local repository is user_home/m2 directory.
- **Central Repository:**
- If any dependency is not found in the local repository, then Maven traverses the central repository. Maven then downloads these dependencies in your local repository.
- **Remote Repository:**
- When Maven wants to download dependency, it goes to a remote repository. A remote repository is a repository present on a web server and is widely used to host the internal projects of an organization.

Maven pom.xml file:

- POM stands for Project Object Model. The pom.xml file contains information about the project and configuration information to build the project.
- It contains dependencies, build directory, source directory, test source directory, plugin, etc. Maven goes through the pom.xml file, then executes the goal.
- For the creation of a simple pom.xml file, you must need the following elements:

- **project** : Root element of a pom.xml file.
- **groupId**: Sub element of the project. It describes the Id for the project.
- **modelVersion**: Sub element of the project. It tells you the model version.
- **artifactId**: Sub element of the project. It specifies the id for the project. An artifact is either produced or used by a project. Examples of artifacts: JARs, source and binary distributions, and WARs.
- **Version**: Sub element of the project. It tells you the version of the artifact under given group.

- There are some additional elements in pom.xml file:
Packaging: defines the packaging type such as war, jar etc.
- **Scope:** defines the scope for the maven project.
- **Url:** specifies the URL of the project.
- **Name:** defines the name of the maven project.
- **Dependency:** defines a dependency. It is used inside dependencies.

Demo

- 1] Download and install JDK
- 2] Set JAVA_HOME and Path upto bin folder in Environmental Variables
- 3] Download Maven
- 4] Set MAVEN_HOME and Path upto bin folder in Environmental Variables
- 5] Verify versions

- 6] Create Maven Project with maven commands--
 - -- Open cmd prompt
 - -- Create a directory
 - - Run command-
 - mvn archetype:generate
 - -DgroupId=<Package Name>
 - -DartifactId=<Project Name>
 - -DarchetypeArtifactId=maven-archetype-quickstart
 - -DarchetypeVersion=1.0
 - -DinteractiveMode=false

- -- Run tests with
 - `mvn test`
 - -- Build project with
 - `mvn package`
-
- Run App.java using that JAR file
 - `java -cp target/INT331-1.0-SNAPSHOT.jar
qa.automation.App`

Module	Description
<u>maven-archetype-plugin</u>	Archetype Plugin to use archetypes with Maven,
<u>archetype-packaging</u>	Archetype lifecycle and packaging definition,
<u>archetype-models</u>	Descriptors classes and reference documentation,
<u>archetype-common</u>	Core classes,
<u>archetype-testing</u>	Components used internally to test Maven Archetype,